qual_by_position.py 사용 설명서

1. 개요

lab11/src/qual_by_position.py 는 FASTQ 파일의 모든 read 에서 같은 위치(1번째 염기, 2번째 염기, …)의 품질 점수를 모아 위치별 평균을 계산한다. 결과는 위치<TAB>평균 품질 형식으로 한 줄에 한 위치씩 출력한다. FastQC 보고서의 Per base sequence quality 그래프에서 평균선이 나타내는 값과 같다.

2. 실행 방법

```
cd lab11
python3 src/qual_by_position.py <FASTQ 파일>
python3 src/qual_by_position.py <FASTQ 파일> > 결과.tsv
```

- 인자는 FASTQ 파일 경로 하나만 받는다.
- 결과는 화면(표준 출력)으로 나온다. 파일로 저장하려면 > 를 쓴다. 같은 이름의 파일이 있으면 덮어쓰므로 먼저 확인한다.
- 압축 파일(.fq.gz)은 바로 읽지 못한다. 먼저 gunzip -k 로 푼다.
- 외부 라이브러리 없이 파이썬 표준 기능만 쓴다.
- 품질 문자는 Phred+33 인코딩(현재 Illumina 표준)이라고 가정한다.

3. 출력 형식

열	내용
1열	위치 (1 부터 시작, FastQC 와 같음)
2열	그 위치의 평균 품질 (소수점 둘째 자리까지 반올림)

- 머리줄(header)은 없다. 줄 수는 가장 긴 read 의 길이와 같다.
- 열 사이는 탭(\t) 하나로 구분한다.

4. 계산 방법: 반복문 안의 반복문

read 마다 길이가 다를 수 있다. 이 스크립트는 위치마다 그 위치를 가진 read 수(count)를 따로 세는 방법을 쓴다. 가장 짧은 read 길이까지만 계산하는 방법보다 버리는 염기가 없다.

```
바깥 반복: read 하나씩           → read 개수(N) 만큼
   안쪽 반복: 그 read 의 위치 i → 그 read 의 길이만큼
        total[i] += 점수
        count[i] += 1
마지막 반복: 위치 하나씩         → 가장 긴 read 길이만큼
   평균[i] = total[i] / count[i]
```

전체 반복 횟수는 모든 read 길이의 합(= 전체 염기 수) 이다.

길이가 다른 read 3개로 손으로 계산한 예:

	위치 1	위치 2	위치 3	위치 4
r1 IIHH	40	40	39	39
r2 I#	40	2		
r3 III	40	40	40	
total	120	82	79	39
count	3	3	2	1
평균	40.0	27.33	39.5	39.0

```
$ python3 src/qual_by_position.py small.fastq
1       40.0
2       27.33
3       39.5
4       39.0
```

5. 실행 예시 (lab11 데이터)

```
$ python3 src/qual_by_position.py data/SRR1553607_1.fastq
1       32.04
2       32.17
3       32.28
4       35.85
5       35.65
...
99      29.03
100     28.93
101     28.25
```

- read 203,445 개, 모두 길이 101. 실행 시간 약 2.5 초.
- fastp 로 자른 data/trimmed.fastq 는 길이가 50~101 로 제각각이다. 1번 위치는 188,357 개 read 가, 101번 위치는 171,032 개 read 가 가지고 있다. 이런 파일에서 위치별 count 를 따로 세야 한다.

6. FastQC 결과와 비교

FastQC 의 fastqc_data.txt 에서 >>Per base sequence quality 의 Mean 열과 비교했다.

```
unzip -p doc/SRR1553607_1_fastqc.zip '*/fastqc_data.txt' \
  | sed -n '/>>Per base sequence quality/,/>>END_MODULE/p'
```

파일	FastQC 구간 수	최대 차이
SRR1553607_1.fastq	55	0.00000000000001
trimmed.fastq	55	0.00012

- FastQC 는 10번 위치부터 10-11, 12-13 처럼 위치를 묶어서 보여 준다. 그래서 줄 수가 101 이 아니라 55 이다.
- 비교할 때는 묶인 위치들의 점수 합을 read 수 합으로 나눠(가중 평균) 맞췄다.
- 두 파일 모두 사실상 같은 값이다.

7. 코드 설명

read_fastq(filename)

qual_summary.py 의 함수와 같다. 파일을 4줄씩 끊어서 (이름, 서열, 품질) 튜플의 리스트를 돌려준다.

qual_by_position(reads)

1. 빈 리스트 total, count 를 만든다.
2. 바깥 반복으로 read 를 하나씩 꺼낸다.
3. 안쪽 반복으로 품질 문자열의 위치 i 를 하나씩 돈다.
4. 지금까지 본 read 보다 긴 위치가 나오면(i == len(total)) 두 리스트에 0 을 하나씩 붙인다.
5. ord(qual[i]) - 33 을 total[i] 에 더하고, count[i] 에 1 을 더한다.
6. 모든 read 를 본 뒤 total[i] / count[i] 로 위치별 평균 리스트를 돌려준다.

메인 부분

1. sys.argv[1] 의 파일을 read_fastq 로 읽는다.
2. qual_by_position 으로 평균을 구한다.
3. 위치는 i + 1 (1 부터), 평균은 round(..., 2) 로 탭을 사이에 두고 출력한다.

8. 오류가 나는 경우

상황	결과
파일 이름을 주지 않고 실행	IndexError: list index out of range
빈 파일	오류 없이 아무것도 출력하지 않음
파일 끝에 빈 줄이 더 있음	4줄 묶음이 어긋나 IndexError 가 나거나 값이 틀어짐
서열이 여러 줄로 나뉜 FASTQ	4줄 고정이라고 가정하므로 잘못 읽음
Phred+64 인코딩(오래된 Illumina)	오류는 없지만 점수가 31 씩 높게 나옴
아주 큰 파일	readlines() 가 파일 전체를 메모리에 올리므로 느리거나 메모리가 부족할 수 있음

- 서열과 품질 문자열의 길이가 같은지는 확인하지 않는다.
- 평균만 계산한다. FastQC 그래프의 상자(사분위수, 10/90 백분위수)는 계산하지 않는다.

부록. 위치별 평균 품질 그래프

lab11 데이터 두 개에 qual_by_position.py 를 실행한 결과이다. FastQC 의 Per base sequence quality 그래프의 평균선에 해당한다. 배경의 초록/노랑/빨강 띠는 FastQC 와 같은 기준(Q28 이상, Q20~28, Q20 미만)이다.

